Universidad Nacional Autónoma de México

Facultad de Ingeniería

Defensas preventivas de los sistemas operativos ante ataques informáticos

---

Semestre: 2026-2
Materia: Sistemas Operativos
Profesor: Gunnar Eyal Wolf Iszaevich
Grupo: 7
Fecha de entrega: 16 de abril de 2026

EQUIPO:

Nombre	No. Cuenta
Espinosa González Sara Sofia	321268121
Rosete Manzano Karina Lizeth	321032308

Índice

1. Introducción **2**

2. Objetivos específicos **2**

3. Modelo de amenazas y superficie de ataque **2**

3.1. Modelo de amenazas 2

3.2. Vulnerabilidades técnicas en el sistema operativo 3

3.3. Superficie de ataque 4

4. Clasificación de defensas preventivas **4**

4.1. Seguridad en capas (Defense in Depth) 5

4.2. El factor humano en la seguridad 5

5. Mecanismos de defensa preventiva **5**

5.1. Autenticación y control de acceso 5

5.1.1. Seguridad en las credenciales 5

5.1.2. Métodos de implementación, el Modelo UGO 6

5.2. Modo usuario y modo kernel 6

5.3. Aislamiento de procesos y memoria virtual 7

5.3.1. Funcionamiento técnico: MMU y tablas de páginas 7

5.4. Aleatorización del espacio de direcciones (ASLR) 8

5.5. Sandboxing 8

5.5.1. Ejemplos técnicos de implementación 8

5.6. Cifrado de información 9

5.7. Actualizaciones del sistema 9

6. Estudio de caso: La defensa en profundidad en acción **9**

7. Limitaciones de las defensas preventivas **10**

8. Conclusión **10**

1. Introducción

En la actualidad, los sistemas operativos no solo administran los recursos de una computadora y coordinan la ejecución de procesos, sino que también representan la principal barrera de protección frente a amenazas informáticas. Su papel central dentro del sistema los convierte en un objetivo constante para atacantes que buscan comprometer la información y el funcionamiento del entorno digital.

En este contexto, la seguridad en los sistemas operativos no debe limitarse a la detección o reacción ante ataques, sino que debe enfocarse en la implementación de mecanismos preventivos que reduzcan la probabilidad de intrusiones. Estas defensas permiten anticipar posibles amenazas y minimizar vulnerabilidades. No obstante, es importante considerar que no todas las medidas de seguridad se implementan a nivel del sistema operativo. Muchas resultan más efectivas en el espacio de usuario.

El presente trabajo tiene como objetivo analizar los principales mecanismos de defensa preventiva en sistemas operativos, abordando su fundamento teórico, funcionamiento técnico y relación con los conceptos fundamentales de la materia.

2. Objetivos específicos

- Analizar los principales mecanismos de seguridad en sistemas operativos.
- Identificar las amenazas más comunes que afectan a estos sistemas.
- Explicar el funcionamiento técnico de las defensas preventivas.
- Relacionar los mecanismos de seguridad con su aplicación en sistemas reales.

3. Modelo de amenazas y superficie de ataque

3.1. Modelo de amenazas

El modelo de amenazas es un enfoque estructurado que permite identificar, analizar y evaluar los posibles riesgos de seguridad en un sistema informático. Este concepto engloba distintas metodologías cuyo objetivo es formalizar qué activos deben protegerse, qué amenazas pueden afectarlos y de qué manera podrían materializarse dichos ataques.

A través de este enfoque, es posible anticipar cómo un atacante podría comprometer el sistema, identificando vulnerabilidades y antes de que estos sean explotados.

Los modelos de amenazas se basan en la identificación de diversos elementos, entre los que destacan los siguientes:

- **Activos:** recursos que deben protegerse (datos, procesos, memoria)
- **Amenazas:** acciones potenciales que pueden comprometer la seguridad.
- **Vulnerabilidades:** debilidades del sistema que pueden ser explotadas.

Desde la perspectiva de los sistemas operativos, el modelo de amenazas permite diseñar mecanismos de protección desde etapas tempranas, integrando la seguridad en la gestión de memoria, procesos y control de acceso.

3.2. Vulnerabilidades técnicas en el sistema operativo

Es importante distinguir entre el software malicioso como agente externo y las vulnerabilidades estructurales que este intenta explotar. Si bien amenazas como el malware o el phishing representan riesgos críticos en la superficie de ataque, las amenazas directas son aquellas fallas en la lógica de control, gestión de memoria y asignación de privilegios que comprometen la integridad del sistema.

- **Buffer overflow:** ocurre cuando un programa escribe más datos en un búfer de longitud fija de los que puede almacenar, permitiendo sobrescribir ubicaciones de memoria. Esto puede ayudar a un atacante para ejecutar código arbitrario con los privilegios del proceso expuesto.
- **Escalación de privilegios:** consiste en la oobtención de permisos superiores a recursos proegidos para una aplicación o usuario, comprometiendo la seguridad del sistema.
- **Condiciones de carrera:** se producen cuando el comportamiento del sistema depende de la sincronización de eventos no controlados, permitiendo que varios procesos accedan a datos compartidos simultáneamente sin que haya exclusión mutua.
- **Denegación de servicio (DoS):** tiene como objetivo saturar el sistema o explotar errores que provoquen su fallo, haciendo que los recursos no estén disponibles para los usuarios.
- **Vulnerabilidades de día cero (Zero-day):** se refieren a fallos de seguridad desconocidos para el proveedor del software que aún no han sido parcheados. Representan un riesgo alto al no existir una defensa directa inmediata hasta que se identifique el fallo y se publique una corrección.

3.3. Superficie de ataque

La superficie de ataque se define como el conjunto de vulnerabilidades, interfaces y puntos de entrada que pueden ser explotados por un atacante para comprometer un sistema.

Se clasifica en tres categorías principales:

- **Superficie digital:** incluye software, sistemas operativos, redes y servicios expuestos.
- **Superficie física:** comprende dispositivos y hardware accesible del equipo, incluyendo periféricos, puertos de comunicación.
- **Superficie de ingeniería social:** involucra la manipulación del usuario para evadir controles técnicos, como la ejecución de códigos desconocidos.

Tipo de superficie	**Ejemplos**
Digital	Contraseñas débiles, software vulnerable, servicios expuestos, puertos de red abiertos
Física	Robo de dispositivos, acceso no autorizado a puertos USB o a dispositivos de almacenamiento sin cifrar,
Ingeniería social	Phishing, manipulación de usuarios, instalación de software de fuentes no confiables

Cuadro 1: Clasificación de la superficie de ataque

En los sistemas operativos, la superficie de ataque se relaciona directamente con la cantidad de procesos, servicios y recursos expuestos, lo que hace fundamental la implementación de mecanismos que reduzcan estos puntos de vulnerabilidad.

4. Clasificación de defensas preventivas

Los mecanismos de seguridad en sistemas operativos pueden clasificarse según su función:

- **Seguridad de acceso:** gestiona la identidad y los permisos mediante mecanismos de autenticación y listas de control (ACLs).
- **Seguridad de ejecución:** Asegura que los procesos operen en entornos controlados mediante la separación de modos (usuario/kernel) y el aislamiento de procesos.
- **Seguridad de memoria:** protege el espacio de direcciones de cada proceso, implementando técnicas como la segmentación y la aleatorización (ASLR).

- **Seguridad de datos:** garantiza la confidencialidad de la información mediante el uso de algoritmos de cifrado y funciones hash.

Esta clasificación permite comprender cómo los distintos mecanismos trabajan en conjunto bajo el principio de seguridad en capas.

4.1. Seguridad en capas (Defense in Depth)

El diseño de seguridad en los sistemas operativos modernos no se basa en una única barrera, sino en el principio de seguridad en capas (también conocido como Defense in Depth). Este enfoque consiste en la implementación de múltiples niveles de protección independientes entre sí. Su objetivo fundamental es garantizar que, si un atacante logra vulnerar una capa específica, existan otras barreras que impidan el acceso total a los recursos, eliminando así los puntos únicos de fallo y aumentando la resiliencia del sistema.

4.2. El factor humano en la seguridad

A pesar de la implementación de múltiples mecanismos de defensa en los sistemas operativos, el usuario continúa siendo uno de los elementos más vulnerables en la seguridad informática. Errores humanos como el uso de contraseñas débiles, la descarga de software de fuentes no confiables o la falta de actualización del sistema pueden comprometer la protección del equipo.

Asimismo, las técnicas de ingeniería social, como el phishing, aprovechan la confianza o desconocimiento del usuario para obtener acceso a información sensible, sin necesidad de vulnerar directamente los mecanismos técnicos del sistema.

Por lo tanto, la seguridad no depende únicamente de las herramientas tecnológicas, sino también de las prácticas y el nivel de conciencia del usuario en el manejo de la información.

5. Mecanismos de defensa preventiva

5.1. Autenticación y control de acceso

La seguridad en el acceso se divide en dos fases: la identificación (donde el usuario declara quién es) y la autenticación (donde el sistema verifica dicha identidad mediante una credencial). Una vez que el usuario es validado, el sistema operativo le otorga los derechos y permisos específicos asociados a su cuenta para iniciar la sesión.

5.1.1. Seguridad en las credenciales

Dado que la contraseña es la principal barrera de protección, los sistemas operativos no las almacenan en texto plano. Estas se cifran mediante funciones hash y se resguardan en

archivos con acceso restringido, como `/etc/security/passwd` o `/etc/shadow`. El acceso a estos archivos está limitado exclusivamente al usuario.

5.1.2. Métodos de implementación, el Modelo UGO

En sistemas tipo Unix (como Linux o macOS), el control de acceso se rige por el modelo de tres columnas denominado UGO (*User, Group, Other*). Este esquema define la relación entre la identidad del sujeto y sus capacidades:

- **User (u):** representa al usuario propietario del archivo.
- **Group (g):** usuarios pertenecientes al grupo asignado al archivo. Es importante destacar que un usuario puede pertenecer a uno o más grupos simultáneamente, permitiendo una gestión colaborativa flexible.
- **Other (o):** usuarios que no son el dueño ni forman parte del grupo asignado.
- Permisos para controlar el acceso a archivos:
  - **r (read):** lectura
  - **w (write):** escritura
  - x (execute): ejecución

Interpretación técnica de permisos (Ejemplo 755) Los permisos se definen mediante la suma de valores octales para cada columna: lectura (**4**), escritura (**2**) y ejecución (**1**)].

Identidad (UGO)	**Valor Octal**	**Simbólico**	**Facultades**
User (Propietario)	7 (4+2+1)	`rwx`	Lectura, escritura y ejecución.
Group (Grupo)	5 (4+0+1)	`r-x`	Lectura y ejecución únicamente.
Other (Otros)	5 (4+0+1)	`r-x`	Lectura y ejecución únicamente.

Cuadro 2: Interpretación del permiso 755 (`rwxr-xr-x`).

Bajo este esquema, el propietario mantiene control total, mientras que los demás sujetos solo pueden consultar o ejecutar el recurso sin modificarl

5.2. Modo usuario y modo kernel

El sistema operativo divide la ejecución en dos niveles:

- Usuario: entorno restringido donde se ejecutan las aplicaciones de usuario. El procesador impide que el código en este modo acceda directamente al hardware o a direcciones de memoria reservadas para el kernel.

- Kernel: nivel con mayores privilegios, donde el núcleo del sistema operativo tiene acceso irrestricto a todas las instrucciones de la CPU y a la totalidad de la memoria física. En este modo se ejecutan funciones críticas como la gestión de interrupciones, el manejo de memoria y el control de dispositivos de hardware.

Esta separación evita que aplicaciones maliciosas comprometan el sistema completo, relacionándose directamente con la gestión de procesos.

La transición entre modo usuario y modo kernel se realiza mediante llamadas al sistema (system calls), las cuales permiten a los programas solicitar servicios del sistema operativo de manera controlada. Esto evita accesos directos al hardware, reduciendo riesgos de seguridad y manteniendo el control por parte del sistema operativo.

5.3. Aislamiento de procesos y memoria virtual

La memoria virtual permite qa cada proceso ejecutarse en un espacio de direcciones lógico e independiente Este aislamiento es fundamental porque garantiza que, aunque un programa sea malicioso o contenga errores, no pueda interferir con la estabilidad de otros procesos ni del núcleo (kernel).

5.3.1. Funcionamiento técnico: MMU y tablas de páginas

El aislamiento es operado mediante la interacción entre estructuras de datos y hardware especializado:

1. **Mapeo Privado:** El kernel gestiona una **tabla de páginas** exclusiva para cada proceso, la cual funciona como un mapa de las únicas direcciones físicas de RAM que dicho proceso puede ver.

2. **Traducción y Validación:** La MMU intercepta cada solicitud de memoria y traduce la dirección virtual usando solo la tabla del proceso activo.

3. **Excepción de Hardware:** Si un proceso intenta acceder a una dirección fuera de su mapa (memoria de terceros), la MMU detecta que no existe entrada válida y genera un *trap* o *segmentation fault*. El kernel toma el control y termina el proceso para proteger el sistema.

5.4. Aleatorización del espacio de direcciones (ASLR)

ASLR es una técnica que introduce un desplazamiento aleatorio (offset) en la dirección de inicio de segmentos clave como el stack, el heap y las bibliotecas dinámicas. Contrario a un intercambio de secciones, esta variabilidad asegura que cada ejecución del programa presente direcciones lógicas distintas.

- **Kernel ASLR (KASLR):** es un ejemplo de implementación o una variante específica de la técnica. Mientras que el ASLR estándar protege a las aplicaciones, el KASLR es el ejemplo de cómo el sistema operativo aplica esta misma protección a su propio núcleo (kernel) para esconder sus estructuras críticas.
- **Mitigación de exploits:** Más que un ejemplo, es el objetivo primordial o la utilidad práctica del mecanismo. ASLR existe precisamente para mitigar la efectividad de ataques como el stack overflow al hacer que las direcciones de memoria sean impredicibles para el atacante.

5.5. Sandboxing

Permite ejecutar programas en entornos aislados. Un entorno aislado es un espacio controlado donde se ejecutan programas sin que puedan interactuar directamente con el sistema principal. El aislamiento consiste en restringir el acceso del programa a recursos como memoria, archivos o procesos, evitando que cause daños. En contraste, los entornos no aislados son aquellos donde los programas se ejecutan directamente en el sistema, teniendo acceso completo a sus recursos, lo que puede representar un riesgo si el software es malicioso.

5.5.1. Ejemplos técnicos de implementación

Esta técnica se concreta mediante mecanismos específicos del sistema operativo que restringen las capacidades de los procesos:

- **Aislamiento en dispositivos móviles:** android e iOS ejecutan cada aplicación con un identificador de usuario (**UID**) único y un sistema de archivos privado. Esto garantiza que una *app* maliciosa no pueda acceder a los datos de otra por diseño de permisos del sistema.
- **Contenedores (Docker):** Utilizan primitivas del kernel como **Namespaces** para aislar la visibilidad de recursos (red, procesos) y **Cgroups** para limitar el consumo de hardware, evitando que un proceso afecte al sistema principal.

- **Navegadores:** Implementan aislamiento por pestañas para asegurar que el código de un sitio web no pueda escalar privilegios hacia el sistema de archivos local o interceptar datos de otras sesiones abiertas.

5.6. Cifrado de información

El cifrado garantiza la confidencialidad de los datos. Los transforma en información ilegible que solo puede ser recuperado mediante una clave autorizada. **Ejemplo:**

- Contraseñas almacenadas como hashes.
- Protección de archivos sensibles.

5.7. Actualizaciones del sistema

permiten corregir fallos y vulnerabilidades que se descubren durante el ciclo de vida del software. Este proceso consiste en aplicar "parches"que cierran los agujeros de seguridad. El sistema operativo verifica que el software provenga de la fuente oficial, evitando así que una actualización falsa introduzca nuevas amenazas en el equipo.

- Reducen riesgos de explotación.
- Mantienen el sistema seguro frente a nuevas amenazas.

6. Estudio de caso: La defensa en profundidad en acción

Para comprender la efectividad real de los mecanismos de seguridad preventivos, es crucial analizarlos no como herramientas aisladas, sino como un sistema integrado de defensa en profundidad. A continuación, se presenta un escenario técnico hipotético para ilustrar cómo estas capas colaboran para neutralizar una amenaza común: la ejecución de código malicioso a través de una vulnerabilidad en un navegador web.

1. **Vulnerabilidad (Entrada):** un usuario visita un sitio web comprometido que explota una falla de **desbordamiento de búfer** en el navegador para intentar ejecutar código dañino.
2. **Capa 1 (Modo de Operación):** el navegador se ejecuta en **modo usuario**, por lo que el código malicioso no tiene acceso directo al hardware ni a la memoria del sistema operativo. Está .[en]cerrado.[en] el anillo de menor privilegio.

3. **Capa 2 (Aislamiento de Memoria):** el ataque intenta saltar a otra zona de la memoria para robar datos de una pestaña diferente. La **MMU** detecta el acceso a una dirección de memoria no autorizada y genera un error, cerrando el proceso del navegador antes de que el robo de información se concrete.

4. **Capa 3 (ASLR):** si el ataque intenta ejecutarse de nuevo, el mecanismo **ASLR** habrá aleatorizado las direcciones de inicio de las bibliotecas del sistema. Esto hace que el atacante no pueda predecir la ubicación exacta de las funciones necesarias para retomar el control de la ejecución.

5. **Capa 4 (Sandboxing):** incluso si el atacante lograra controlar el proceso del navegador, este opera dentro de un **sandbox**. El entorno aislado le impide al proceso malicioso escribir archivos nuevos en el disco duro o acceder a los archivos personales del usuario, conteniendo así el impacto del ataque.

Este escenario demuestra que, aunque una capa pueda ser vulnerada o eludida, la presencia de barreras redundantes e independientes incrementa significativamente la robustez estructural del sistema operativo frente a amenazas complejas.

7. Limitaciones de las defensas preventivas

A pesar de los mecanismos existentes:

- No existe un sistema completamente seguro.
- Existen vulnerabilidades desconocidas (zero-day)
- Factor humano: el usuario sigue siendo un punto crítico de fallo

8. Conclusión

Las defensas preventivas en sistemas operativos representan un elemento fundamental para garantizar la seguridad de los sistemas informáticos. A través de mecanismos como el control de acceso, el aislamiento de procesos y la protección de memoria, es posible reducir significativamente la probabilidad de intrusiones. Asimismo, el enfoque de seguridad en capas permite combinar múltiples estrategias para fortalecer la protección del sistema. Sin embargo, la seguridad no es un estado absoluto, sino un proceso continuo que requiere actualización constante y una adecuada gestión de riesgos. Asimismo, el avance constante de las amenazas informáticas obliga a que los sistemas operativos evolucionen continuamente en sus mecanismos de defensa. La seguridad no solo depende de la tecnología implementada, sino también

del usuario, quien representa un factor crítico en la protección del sistema. La ausencia o debilidad en los mecanismos de seguridad de un sistema operativo puede tener consecuencias graves, como la pérdida de información confidencial, accesos no autorizados, interrupciones en los servicios y daños económicos significativos. En contextos más críticos, como sistemas financieros o de salud, una vulnerabilidad puede comprometer incluso la integridad de las personas. Sin embargo, muchos de estos mecanismos pueden ser insuficientes frente a ataques avanzados si no se implementan correctamente o si el sistema no se mantiene actualizado, lo que evidencia la necesidad de una seguridad en constante evolución. Esto demuestra que la seguridad en los sistemas operativos no es un aspecto opcional, sino un elemento fundamental para garantizar el correcto funcionamiento de los sistemas digitales en la actualidad.

Referencias

- National Institute of Standards and Technology. (2020). *Security and privacy controls for information systems and organizations (SP 800-53).*https://www.nccoe.nist.gov/publication/1800-26/VolA/index.html
- Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson.
- Stallings, W. (2018). *Operating systems: Internals and design principles* (9th ed.). Pearson.
- Ibm. (2025, 26 noviembre). Superficie de ataque. IBM. https://www.ibm.com/mx-es/think/topics/attack-surface
- Universidad de Buenos Aires. (s. f.). *Modelado de amenazas y análisis de riesgos en sistemas informáticos.* https://criptored.es/cibsi/cibsi2009/docs/Papers/CIBSI-Dia2-Sesion3(2).pdf
- Banco Santander.(s.f.). *Qué es una vulnerabilidad informática - Banco Santander.* https://www.bancosantander.es/glosario/vulnerabilidad-informatica
- Spektor, H. (2024, 28 octubre). *Operating System Vulnerabilities: Understanding and Mitigating the Risk. Sternum IoT.* https://sternumiot.com/iot-blog/operating-system-vulnerabilities-understanding-and-mitigating-the-risk/
- AprendeTIC UNAM. (2022, 17 noviembre). *Modelado de amenazas y su impacto en la seguridad informática de la organización.*https://www.youtube.com/live/ZU27OYUU8jU?si=ak6Dd2rF7JYmFOmU
- C, G. G. (2021, 1 febrero). *Ejemplos explicados de permisos en Linux - Comandos básicos y avanzados.* Naps Tecnología y Educación. https://naps.com.mx/blog/ejemplos-explicados-de-permisos-en-linux/
- Administración de Memoria: Memoria Virtual. (s.f.). https://www.fing.edu.uy/tecnoinf/mvd/cursos/so/material/teo/so09-memoria_virtual.pdf